Код:https://github.com/SkakunOstap/CryptographyHW3

Пункт 1. Реалізація - *sha256.py*

Кодова реалізація написана на Python, використовуючи структури bytearray для роботи з байтами тексту та стану

Тестування - *sha256_test.py*

Тестові значення:

- “abc”
- “”
- “abcdbcdecdefdefgefghfghighijhijkijkljklmklmnlmnomnopnopq”
- “abcdefghbcdefghicdefghijdefghijkefghijklfghijklmghijklmnhijklmnoijklmnopjklmnopqklmnopqrlmnopqrsmnopqrstnopqrstu”
- повторення “a” 10^6 разів

Результат: успішний

Пункт 2. Підбір префіксу - *sha256_prefix_find.py*

Префікс: “000000000000001.b7e-”
Перевірка на 32 біти ідентична перевірці на перші вісім значень hex = 0 (кожен з 0x00 кодується чотирма нульовими бітами)

Пункт 3. Ключі та сертифікат

Згенерували приватний RSA-ключ на 8192 біти

openssl genpkey -algorithm RSA -out server.key -pkeyopt rsa_keygen_bits:8192

Далі сформували запит на сертифікат CSR - це доказ, що той, хто подає запит, реально володіє відповідним приватним ключем.

```
openssl req -new -key server.key -out server.csr \
  -subj "/C=UA/L=Kyiv/O=KSE/CN=www.crypto.kse.ua
emailAddress=user@kse.org.ua" \
  -nodes
```

Перевірили CSR командою “openssl req -text -noout -in server.csr” отримали Subject, публічний ключ 8192 біт з експонентою 65537, самопідпис sha256WithRSAEncryption.

Створили власне сертифікат, підписавши CSR тим самим ключем:

```
openssl x509 -req -sha256 -in server.csr -signkey server.key -out server.crt
```

В отриманому server.crt Issuer і Subject збігаються – обидва вказують на наш CN (www.crypto.kse.ua)

```
Subject: CN=www.crypto.kse.ua emailAddress=user@kse.org.ua
Issuer:  CN=www.crypto.kse.ua emailAddress=user@kse.org.ua
Serial:  1a:08:bd:4d:61:96:19:4d:0e:07:cd:4e:99:63:b5:bf:c9:e0:fc:ce
SKI:     6B:9A:B4:4D:D8:C2:3F:DA:E0:3F:7D:18:75:AC:60:4C:85:AD:87:49
```

Пункт 4. Перехресний підпис з групою #8

За списком наша пара - група #8 (Alina Myshkovska, Nikita Bezverkhyi). CSR можна сміливо передавати у відкритому вигляді, бо там нічого секретного немає, тільки публічний ключ і Subject.

Передано групі #8 : ‘server.csr’ (наш CSR)
Отримано від групи #8: ‘server_8.csr’ (їхній CSR)

Перш ніж щось підписувати – перевірили, що саме нам дають:

```
openssl req -text -noout -verify -in server_8.csr
```

Перший рядок - Certificate request self-signature verify OK, тобто файл цілий, і вони володіють відповідним приватним ключем. Subject — C=UA, L=Kyiv, O=KSE, CN=user@kse.org.ua. Жодних дивних доменів. Можна підписувати.

```
openssl x509 -req -sha256 -in server_8.csr \
  -CA server.crt -CAkey server.key -CAcreateserial \
  -out server_8_signed_by_us.crt -days 365
```

Результат:
- Certificate request self-signature verify OK - підпис цілісний
- Subject: C=UA, L=Kyiv, O=KSE, CN=user@kse.org.ua

В отриманому сертифікаті Issuer – наш Subject, Subject – їхній, прив'язка є. Файл передали їм.

Назад прийшов team_1_signed.crt – наш сертифікат, підписаний їхнім ключем. Хотілося переконатися, що вони підписали саме наш публічний ключ, а не якийсь свій. Порівняли SHA-256у нашому CSR і в сертифікаті, який вони повернули:

```
$ openssl req -in server.csr -modulus -noout | sha256sum
acee12f52f3434158a4359ea336ddbb9eeaf68dee99868cf3d30fa1d158b6bff  -
```

```
$ openssl x509 -in team_1_signed.crt -modulus -noout | sha256sum
acee12f52f3434158a4359ea336ddbb9eeaf68dee99868cf3d30fa1d158b6bff  -
```

Хеші збігаються — підмін нема)

Пункт 5. Підпис повідомлення

Записали повідомлення ‘give my friend 2 bitcoins for a pizza' в файл message.txt

Витягли публічний ключ окремим файлом:

```
openssl rsa -in server.key -pubout -out server.pub
```

Перший підпис PKCS#1

Це той варіант, що завдання називає "безпосередньо геш-значення SHA-256". Openssl у dgst -sha256 -sign бере SHA-256 хеш, обгортає його в PKCS#1 padding і шифрує приватним ключем

```
openssl dgst -sha256 -sign server.key -out sig_pkcs1.bin message.txt
openssl dgst -sha256 -verify server.pub -signature sig_pkcs1.bin message.txt =>
Verified OK
```

Другий підпис RSA-PSS

Той самий процес, але з PSS-padding замість PKCS#1. Треаб передати параметр rsa_pss_saltlen:-1 “довжина солі = довжина хешу”

```
openssl dgst -sha256 -sign server.key \
  -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:-1 \
  -out sig_pss.bin message.txt

openssl dgst -sha256 -verify server.pub \
  -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:-1 \
  -signature sig_pss.bin message.txt
```

Знов Verified OK.

Спостереження

Згенерували по два підписи на одному й тому ж повідомленні – PKCS#1 і PSS та порівняли:

```
$ sha256sum sig_pss.bin sig_pss_2.bin
35112b79a4ecb4aa29311644d349ef720a6fa8d78191fd161bf691eba30a215a
sig_pss.bin
34f444968fc201947556072417367a4e4dd6cf5e985f247b029bd272ec31bcb0
sig_pss_2.bin

$ sha256sum sig_pkcs1.bin sig_pkcs1_2.bin
6be1c15c94851167f335dc5c46aabb2a001ede1ca2e8a959df7f6fa30396eb44
sig_pkcs1.bin
6be1c15c94851167f335dc5c46aabb2a001ede1ca2e8a959df7f6fa30396eb44
sig_pkcs1_2.bin
```

PSS – щоразу інший підпис, бо в кожен підпис додається свіжа випадкова сіль. PKCS#1 – детермінований, тому два підписи побайтно ідентичні.

Чому textbook RSA поганий

Детермінізм. Підписав те саме повідомлення двічі - отримав два однакові підписи. Це погано, бо атакувальник може просто запам'ятовувати: “о, цей підпис я вже бачив, значить це знову повідомлення *give my friend 2 bitcoins*”. Замість того щоб розшифровувати щось, він просто впізнає повідомлення за виглядом підпису.

Мультиплікативність головна біда.

s(m_1) * s(m_2) mod n = s(m_1 * m_2 mod n)

Тобто якщо я маю два валідних підписи я можу їх просто перемножити і отримати валідний підпис. Без знання приватного ключа. Просто перемножив два числа.

Чому PSS - добре

Випадкова сіль. Перед підписом до повідомлення додається 32 байти випадковості (salt). Один і той самий лист щоразу підписується по-різному. Ні детермінізму (сіль різна), ні мультиплікативної атаки. Це дефолт у сучасних стандартах.

Пункт 6. RSA-шифрування з key.pub

key.pub – публічний ключ, наданий разом із завданням. Ідея проста: шифруємо повідомлення цим ключем, і коректність нашого шифрування зможе перевірити викладач

Перед шифруванням лише глянули, що це справді 8192-бітний RSA з експонентою 65537 – “openssl pkey -pubin -in key.pub -text -noout” це підтвердив.

Шифрування – стандартною командою openssl без додаткових опцій:

```
openssl pkeyutl -encrypt -pubin -inkey key.pub -in message.txt -out message.enc
```

За замовчуванням pkeyutl -encrypt використовує PKCS#1 v1.5 padding. Результат - файл message.enc рівно на 1024 байти. Розмір шифротексту фіксований і не залежить від довжини plaintext.

Перевірили, чи рандомізує openssl шифрування - зашифрували те саме повідомлення тим самим ключем двічі:

```
$ sha256sum message.enc message.enc.2
3d9d5b0e9c45e47c8b5282c7d5034135b6478a6dc8967547ce46f2c5939ebf95  message.enc
ceefa44d62431735c3e2604f820443938b5759c15c70fb467cf3ac2a6599c4d7  message.enc.2
```

Шифротексти різні. PKCS#1 v1.5 включає випадкові байти PS між 0x00 0x02 і даними: 0x00 || 0x02 || PS || 0x00 || M . Саме тому атакувальник не може просто взяти якесь X, зашифрувати його і порівняти, щоб перевірити “це те саме чи ні”

Файли в директорії

- server.key – наш приватний RSA-8192
- server.csr – наш CSR
- server.crt – самопідписаний сертифікат
- server.pub – наш публічний ключ (витягнутий з server.key для зручності)
- key.pub – наданий чужий ключ для п.6
- server_8.csr – CSR групи #8
- server_8_signed_by_us.crt – сертифікат групи #8, підписаний нами
- team_1_signed.crt – наш сертифікат, підписаний групою #8
- message.txt – повідомлення
- sig_pkcs1.bin, sig_pkcs1_2.bin – два підписи PKCS#1 v1.5

- sig_pss.bin, sig_pss_2.bin – два підписи PSS
- message.enc, message.enc.2 – два шифротексти RSA з PKCS#1 v1.5 padding, це для вас на перевірку

Пункт 7. Дослідження сертифікатів kse.ua

Найдавніший знайдений сертифікат з common name kse.ua.

						Authority,
4725403437	2021-06-19	2021-06-19	2021-09-17	kse.ua	kse.ua www.kse.ua	C=US, ST=TX, L=Houston, O="cPanel, Inc.", CN="cPanel, Inc. Certification Authority"

Дата видачі: 19 червня 2021
Термін дії: 90 днів

Пункт 8. Обчислення відбитку - *sha256_certificate.py*

Обчислення відбувається за допомогою хешування байтового файлу сертифікату.
Покрокове рішення:

- Завантаження діючого сертифікату у .crt розширенні (за допомогою браузера)
- Перетворення файлу у .der розширення за допомогою:
  openssl x509 -in kse.ua.crt -outform der -out certificate.der
- Знаходження хеша за допомогою скрипта

Авторство

Програмна реалізація Sha256, Дослідження сертифікатів kse.ua (1-2, 7-8 пункти): Остап Скакун
Робота з ключами 3 - 6 пунки: Харін Тимофій